EVERY DEVICE. ONE TOOL. PAY ONCE.

DOCUMENTATION — MODBUSMANAGER PRO V1.9.10 · STANDARD V1.4.14 · UPDATED 14 NOVEMBER 2026

MODBUSMANAGER USER MANUAL

How to connect, poll, write, simulate, script, log, alarm, bridge and visualise Modbus data — and read Siemens S7 PLCs — with ModbusManager on Windows. Use the contents to jump to a topic, or download the whole manual as a printable PDF.

Free 14-day trial • Standard $49 • Pro $119 • Runtime $39 per machine • one-time licences • Windows 10 / 11

GETTING STARTED

ModbusManager is a Windows desktop tool for Modbus devices — PLCs, drives, energy meters, inverters and controllers. It polls and writes registers over Modbus TCP, RTU and ASCII, answers as a slave simulator, and in the Pro edition adds tags with Structured Text, a drag-and-drop HMI dashboard, a Historian, alarms, a gateway and Siemens S7 access. Everything runs locally; no cloud account is needed.

EDITIONS

EDITION	WHAT IT INCLUDES
Standard — $49	Connecting (all modes), poll windows and the register table, Write, Chart, Serial Monitor, Test Center, Slave Scan, Address Scan, the slave simulator, workspaces. Sections marked `STD + PRO` apply to it.
Pro — $119	Everything in Standard, plus Dashboard, Tags & Structured Text, History, Alarms, Gateway, Siemens S7, Runtime packages, device templates, Auto-detect, Diagnose, register snapshots and the PDF commissioning report. Marked `PRO`.
Runtime — $39 / machine	Runs a dashboard package made in Pro full screen on an OEM or panel PC, with no editor. See Runtime edition.

INSTALL 1

RUN THE INSTALLER

Download the installer and start ModbusManager from the Start menu.

TRIAL 2

14 DAYS, FULL FEATURES

The licence button in the toolbar shows the days left, e.g. “Trial: 12d”.

CONNECT 3

ADD YOUR DEVICE

Press **Connect** (or `F3`), choose the mode, enter the parameters.

BUILD 4

POLL & VISUALISE

Read registers, name them as tags, then build an operator screen.

On the first start Pro asks **“What would you like to do?”** — *Read a device*, *Simulate a slave*, *Combine several devices*, *Build a dashboard* or *Just let me look around* — and opens the matching tab. During the trial a short licence reminder appears at start and every 15 minutes; it also asks whether you managed to connect, and *I am stuck* leads to tips and a diagnostic text you can copy into a support e-mail.

WHERE TO GO NEXT

GUIDE

HMI DASHBOARD

Build a live operator screen without SCADA licences.

GUIDE

SIEMENS S7 PUT/GET

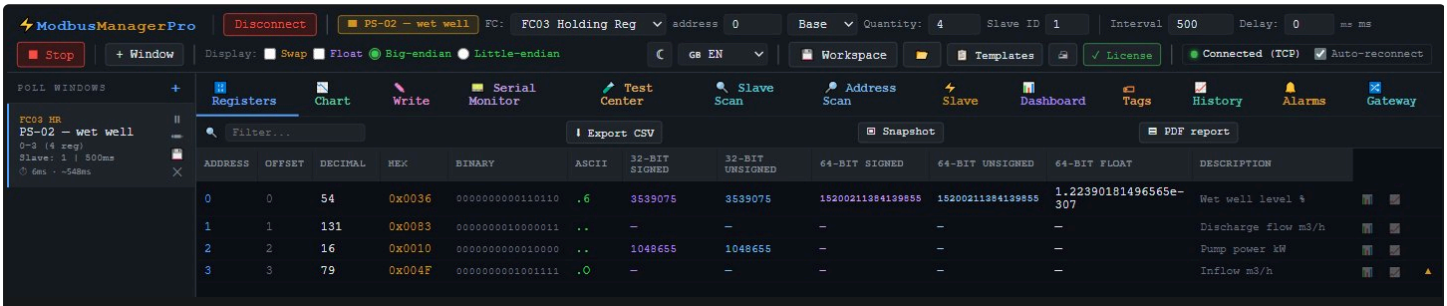
Read and write an S7-1200/1500 with no PLC code.

PDF

FULL MANUAL

This manual as a printable PDF.

THE MAIN WINDOW



Toolbar, poll windows on the left, the tabs, and the Registers table polling a pump station over Modbus TCP.

TOOLBAR

- **Connect / Disconnect** opens the connection dialog (`F3`).
- The amber badge names the selected **poll window**. **FC**, **address**, **Base 0 / Base 1**, **Quantity**, **Slave ID** and **Interval** belong to that window. **Delay** is a global pause between successive requests.
- **▶ Start / ■ Stop** starts and stops polling; **+ Window** adds a poll window (`Ctrl`+`M`).
- **Display: Swap**, **Float**, **Big-endian** / **Little-endian** change how 32-bit values are shown in the table.
- The moon/sun button switches the dark and light theme; the flag selects the language.
- **Workspace** (save) and the folder button (open), **Templates** `PRO`, Print, and **License**, which also shows the licence state: “Trial: 12d”, “✓ License” or “License expired”.
- On the right: the connection state and the **Auto-reconnect** switch.

POLL WINDOWS

Each poll window is one block of registers (function code, start, quantity, slave ID, interval). Every enabled window is polled in the background, whichever one is selected — so one meter's voltages and another device's status can be read side by side. In the list each window has pause/resume, edit (✎), save to the **Saved** list, and remove (×).

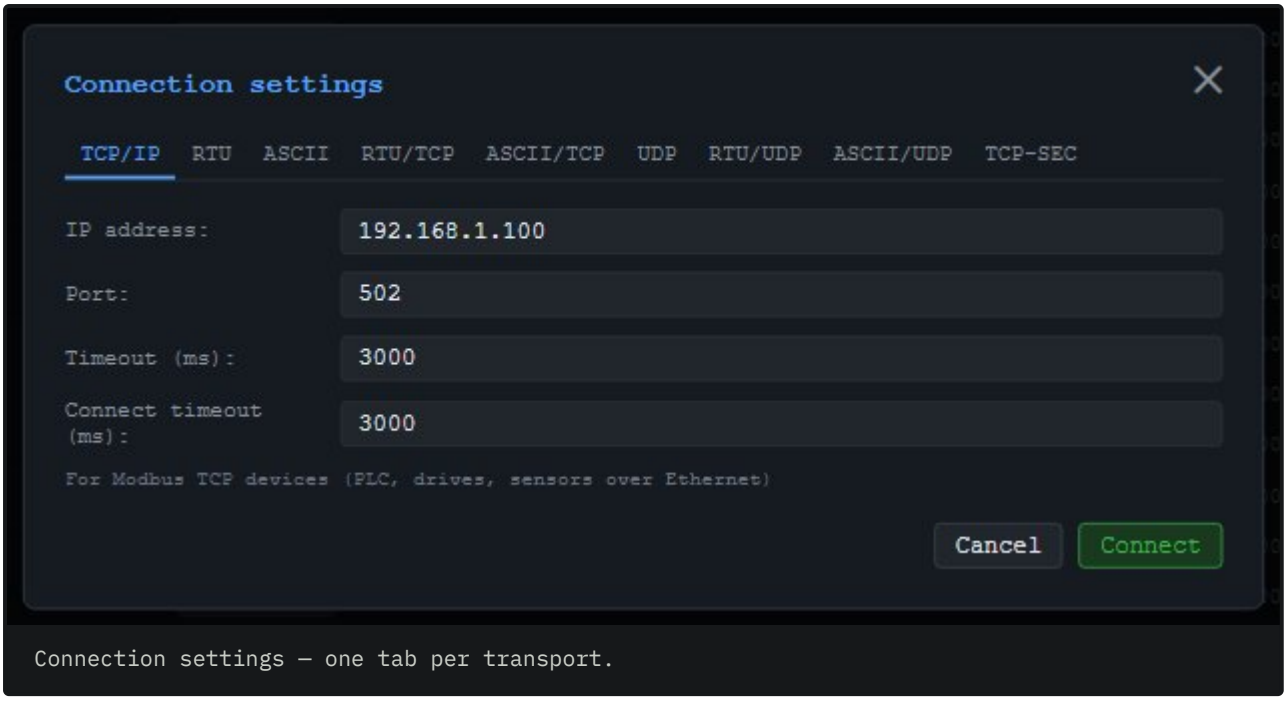
TABS

Registers, Chart, Write, Serial Monitor, Test Center, Slave Scan, Address Scan, Slave `STD + PRO` · **Dashboard, Tags, History, Alarms, Gateway** `PRO`.

LOG, STATUS BAR, TRAY

The **TX/RX log** under the tabs shows every request and reply (Stop pauses it, Clear empties it). The status bar counts TX, RX and **Errors**, shows the **Latency** and the device address, and has the 🔍 **Diagnose** link `PRO`. The tray icon menu offers *Keep running when window is closed* and *Start with Windows* — used by the gateway and alarms when they must run without a window. **Help** → **User manual** opens this page.

CONNECTING STD + PRO



Connection settings — one tab per transport.

MODE	USE IT FOR
TCP/IP	Modbus TCP devices on Ethernet. Fields: IP address, Port (502), Timeout, Connect timeout.
RTU	Serial RS-485 / RS-232 through a COM port or USB adapter.
ASCII	Serial Modbus ASCII (each byte as two hex characters). Port, baud (up to 19200) and parity.
RTU/TCP	RTU frames (with CRC, no MBAP header) through a TCP socket — serial-to-Ethernet converters in transparent mode. Needs Pro 1.9.6 / Standard 1.4.11 or later; earlier versions sent Modbus TCP frames here. More: Modbus RTU over TCP.
ASCII/TCP	Modbus ASCII frames (`:` + hex + LRC) through a TCP socket — converters that pass ASCII through.
UDP	Modbus UDP (the TCP header, in datagrams).
RTU/UDP	RTU frames with their CRC in UDP datagrams.
ASCII/UDP	Modbus ASCII frames in UDP datagrams.
TCP-SEC	Modbus/TCP Security: Modbus TCP inside TLS 1.2 or newer, port 802, with a client certificate — see below.

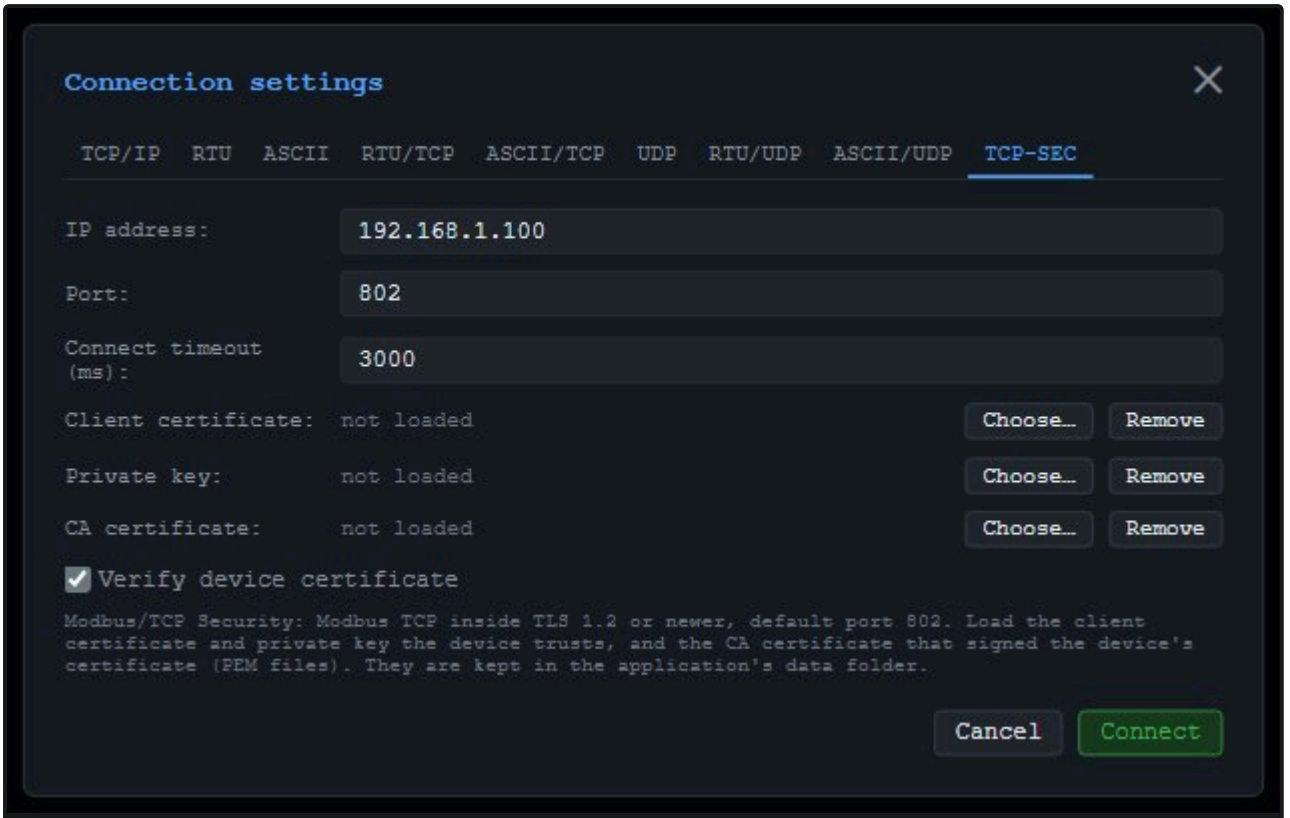
MODBUS TCP

1. Press **Connect**, stay on **TCP/IP**.
2. Enter the IP address and port (usually 502) and press **Connect**.
3. Set the **Slave ID** (unit ID) of the poll window — often 1, sometimes 0 or 255 for a direct TCP device; a gateway uses the ID of the serial device behind it.

MODBUS RTU

1. Plug in the USB-RS485 adapter and open **Connect** → **RTU**. Pick the **Serial port** (↻ refreshes the list).
2. Match the device: **Baud rate** (1200–115200, default 19200), **Parity**, **Data bits**, **Stop bits**. **Custom baud** overrides the list.
3. **RTS Toggle** raises RTS while sending, for RS-485 adapters that switch direction on RTS. Leave it off for adapters with automatic direction control (most USB adapters).
4. **Remove Echo** is for adapters that echo what they send: the echo of the request is dropped before the reply. Leave it off otherwise — on an adapter without echo, single writes (FC05/FC06), whose reply equals the request, would time out.

MODBUS/TCP SECURITY (TCP-SEC)



TCP-SEC: the certificate files and the device check.

1. Open **Connect** → **TCP-SEC**, enter the IP address and port (802).
2. **Client certificate** and **Private key**: the PEM files the device trusts (issued by the device's or your own certificate authority). **Choose…** loads a file once; it is kept in the application's data folder, not in the dialog.
3. **CA certificate**: the certificate that signed the device's certificate. With **Verify device certificate** ticked, a device whose certificate it did not sign is refused — untick it only for a test device with a self-signed certificate.
4. **Connect**. The log names the TLS version; a refusal says why (untrusted device, wrong name, expired, or the device rejected the client certificate).

> NOTE — AUTO-RECONNECT

With **Auto-reconnect** ticked, a dropped connection is retried up to 10 times with growing pauses (2 s up to 15 s); the log shows “Auto-reconnect attempt n/10”.

TIP

Unknown baud rate or slave ID? Use Auto-detect PRO in the Slave Scan tab.

POLL WINDOWS & REGISTERS STD + PRO

FC	READS	TYPE
FC01	Coil Status	Read/write bits
FC02	Input Status	Read-only bits
FC03	Holding Registers	Read/write 16-bit words
FC04	Input Registers	Read-only 16-bit words

- **New poll window** (+ Window or `Ctrl` + `M`): Name, Function, Start address, Quantity (max 125), Slave ID, Interval (ms). In Pro, ↑ **Import CSV** loads a register map with descriptions.
- **Base 0 / Base 1**: Base 0 is protocol addressing (first register = 0), Base 1 documentation addressing (first register = 1). Choose what your device manual uses. A “40001”-style manual usually means Base 1 with FC03.
- **Interval** is per window; **Delay** is a global gap between requests, useful for slow serial devices.

THE REGISTER TABLE

Every value is shown at once as **Decimal, Hex, Binary, ASCII, 32-bit signed/unsigned, 64-bit signed/unsigned and 64-bit float**, plus a **Description**. A value that changed since the last poll is marked ▲.

- **Double-click a value** to write it (FC06/FC05/FC16) — see Writing. **Double-click the description** to name the register.
- **Right-click**: *Copy value*, *Write value*, *Value Names* (show 0 as “Off”, 1 as “Run”…), *Conditional Colors* (colour a cell by a rule). On a 32-bit cell, right-click picks the **word order**: big-endian, word-swapped, byte-swapped or fully reversed.
- Row buttons: 📊 *Add to Dashboard* and 📈 *Log to History* PRO.
- **Filter…** narrows the rows; ⬇ **Export CSV** saves the active window or all windows.
- PRO **Snapshot** stores a baseline and highlights what changed since (**Clear diff** ends it); **PDF report** prints a Modbus commissioning report of the windows.

SCALING

In **Pro** the register table always shows the raw value the device sent. Engineering units belong to tags (Scale and Offset) and to History series, so one register can be scaled once and used everywhere. In **Standard**, right-click → **Scaling** multiplies and offsets the value in the table.

WRITING VALUES STD + PRO

The **Write** tab writes **FC05** single coil, **FC06** single register and **FC16** multiple registers. **FC22** mask write and **FC23** read/write are in the Test Center, which shows the reply. Choose the **Format** (16-bit, 32-bit signed/unsigned/float, 64-bit signed/unsigned/float) and the tool splits the value into registers in the word order you selected. Enter the register address, value (or comma-separated values) and slave ID, then **Send command** →. The reply or the exception is reported at once.

For a quick change, double-click a value in the register table: the **Write Register** dialog opens with the address filled in.

> WARNING
> A write changes the controller. Check the slave ID, the address base and the value before sending — especially on running machinery.

CHART, SERIAL MONITOR & TEST CENTER STD + PRO

- **Chart**: up to three charts, each one register with its slave, format, name and Y range. **Cursor** reads values off the line, and **Log** records the series to a file you can download.
- **Serial Monitor**: the bytes on the wire, as HEX or ASCII with timestamps — on Modbus TCP the whole MBAP frame (transaction ID, unit ID), on RTU one line per frame even when the adapter delivers it in pieces. It keeps running across a reconnect. **Download** saves the capture.
- **Test Center**: compose one request by hand — FC01–06, 15, 16, 22, 23, 08 (diagnostics, with sub-codes), 11 (comm event counter), 17 (report server ID) and 43/14 (device identification) — and inspect the exact reply. FC22 has AND and OR mask fields, FC23 a write address beside the read address and quantity, FC08 a data word next to the sub-code, FC43 the read code (basic, regular, extended or one object) and an object ID; FC17 shows the run status. A value that is not a number is refused, not sent as 0. Without a connection **Send** is off and its tooltip says why. A history keeps the last requests.

SCANNING & DIAGNOSTICS

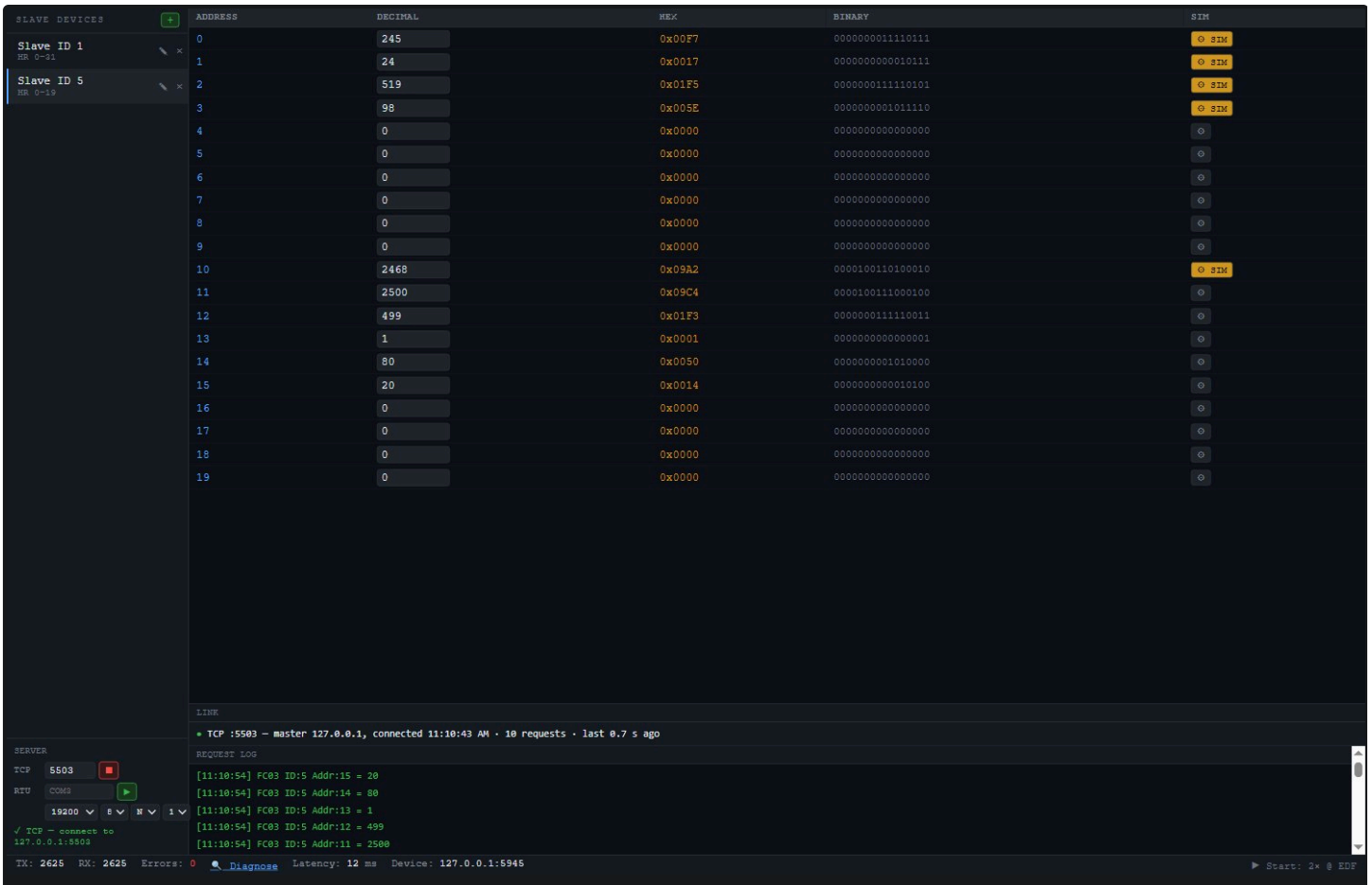
Slave Scan `STD + PRO` probes a range of slave IDs (From ID / To ID) with one read and lists who answers — for a device on the bus whose address is unknown. A device that answers with a Modbus exception counts as found — it is there, and the register or function code is what to change. **Address Scan** sweeps a register range on one slave in blocks and lists which addresses return data, reading address by address where a block has a gap — for mapping an undocumented device; the result exports to CSV.

⚡ **Auto-detect** `PRO`, in the Slave Scan tab, tries baud rates, parity settings and slave IDs on a COM port (or unit IDs behind a TCP gateway) until a device answers, then **Use these settings** applies them.

🔍 **Diagnose** `PRO`, in the status bar, reads the recent error history and says what kind of failure it is: no reply, timeouts now and then, a Modbus exception, refused or reset TCP, garbled frames, port not found or port busy — with what to check for each.

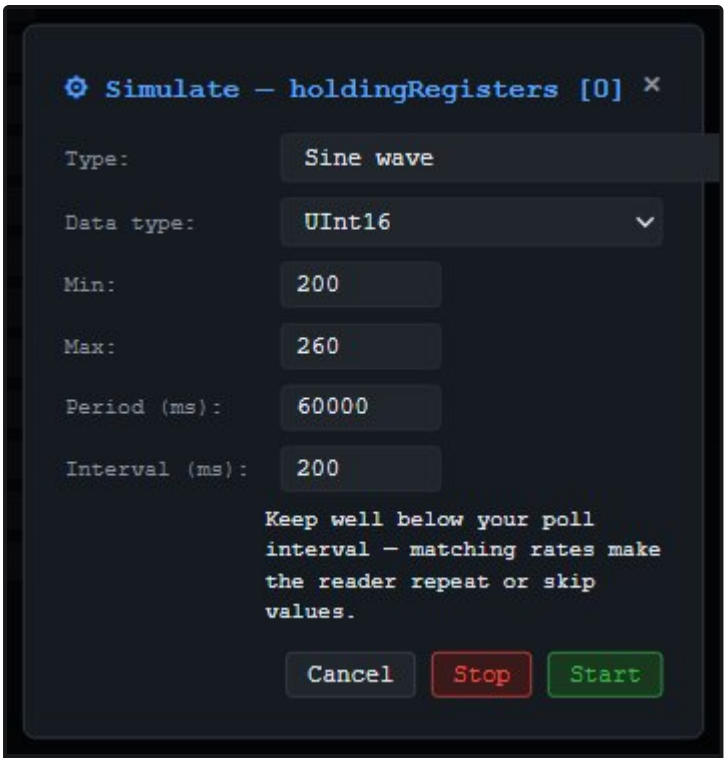
SLAVE SIMULATOR `STD + PRO`

The **Slave** tab makes ModbusManager a Modbus server that answers like real hardware: for testing a SCADA, HMI or PLC program before the device exists, for training, or as the far end of the gateway.



Slave ID 5 with four simulated registers and a PID process at 10–15; the Link panel shows the master that is reading it.

1. + under **Slave devices** adds a block: Slave ID, Register type (Holding, Input, Coils, Discrete Inputs), Start address, Quantity (up to 1000). A slave ID can have several blocks; overlapping blocks are refused.
2. Under **Server** start **TCP** on a port (502 by default) and/or **RTU** on a COM port. The RTU line has its own baud, data bits, parity and stop bits — independent of the Connect dialog, so the same PC can be master on one port and slave on another.
3. Type a value straight into the **Decimal** column, or press the ⚙ button for a simulation.



A sine wave between 200 and 260 with a one-minute period.

Simulation types: **Static value** PRO, **Counter ↑**, **Counter ↓**, **Sine wave**, **Random**, **Ramp** and **PID loop (simulated process)**, each as UInt16, Int16, Int32, UInt32 or Float32 with a word order. Keep the simulation interval well below the master's poll interval, and a sine period at least ten times the poll interval, or the master sees repeated or skipped values.

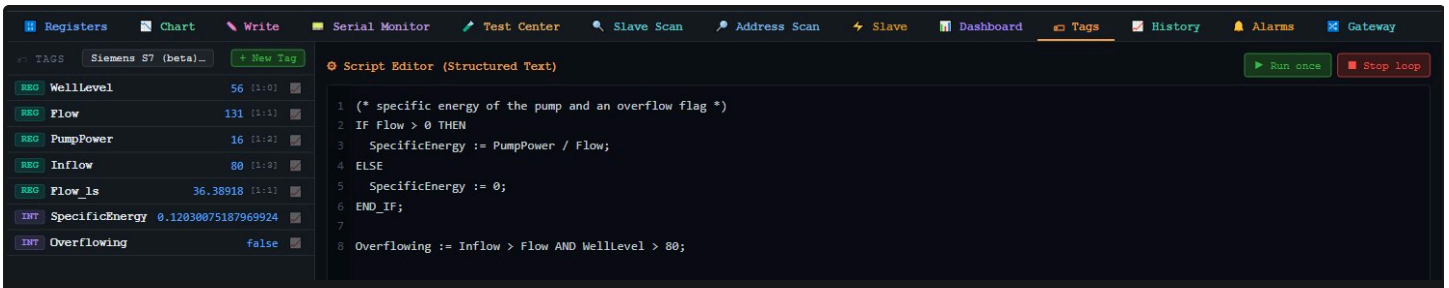
The **PID loop** uses six holding registers: +0 process value ×10, +1 setpoint ×10, +2 output 0–1000, +3 mode (1 = auto), +4 P ×10, +5 I in seconds. A master — or the dashboard's PID faceplate — can change the setpoint and watch the process follow.

LINK PANEL

The **Link** line tells you whether a master is really there: “TCP :502 — master 192.168.0.10, connected … requests · last 0.7 s ago”, the RTU request rate and bad-CRC frames (wrong baud or parity), requests for a unit ID this slave does not have, and refused requests with the reason in the **Request log** below.

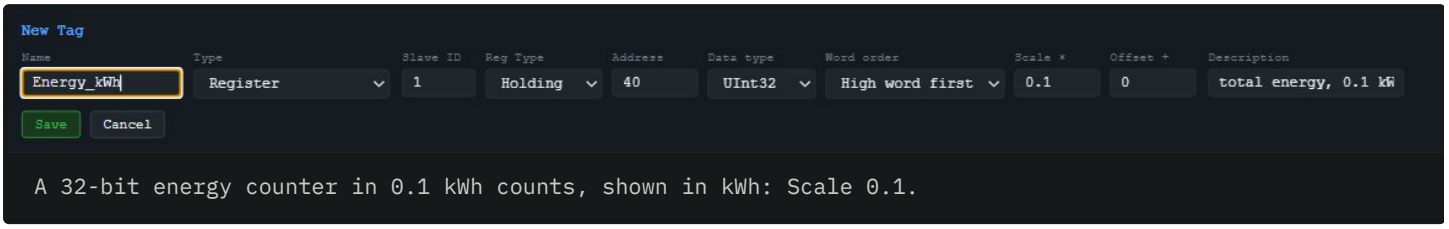
TAGS & STRUCTURED TEXT PRO

A tag gives a value a name, so dashboards, alarms, the Historian, the gateway and scripts can use `WellLevel` instead of “slave 1, FC03, address 0”.



Register tags (REG), a scaled tag (Flow_ls) and internal tags (INT) computed by a Structured Text loop.

TYPE	DESCRIPTION
Register (REG)	A Modbus value: Slave ID, Reg Type, Address, Data type (Int16, UInt16, Int32, UInt32, Float32), Word order (High word first / Low word first), **Scale** × and **Offset** +.
Internal (INT)	A value that lives in the application, usually set by a script.
Siemens S7 (S7)	A value in an S7 PLC: PLC, area (DB, M, I, Q), DB number, byte, bit and type. See Siemens S7.



A 32-bit energy counter in 0.1 kWh counts, shown in kWh: Scale 0.1.

Scale and offset: value = raw × scale + offset. A write to a scaled tag (from the dashboard, a script or the gateway) is converted back to the raw register value. Names start with a letter and contain letters, digits and _; they are not case-sensitive in scripts. Renaming a tag carries its dashboard bindings, alarms and history with it; deleting one tells you first where it is used.

> NOTE — WHERE VALUES COME FROM
>
> A register tag gets its value from polling: its address must be inside a running poll window or bound to a dashboard element, and polling (▶ Start) must run.

STRUCTURED TEXT

The **Script Editor (Structured Text)** runs IEC 61131-3 style code against the tags. ▶ **Run once** tests it; ▶ **Start loop** runs it every **Loop interval** (minimum 100 ms). The output box shows each assignment or the error with its line number. A run that never ends is stopped after 2 s. After five errors in a row the loop pauses 30 s and tries again (then 1, 2, up to 5 minutes), so a device that was away for a while does not leave the script stopped.

```
(* specific energy of the pump and an overflow flag *)
IF Flow > 0 THEN
  SpecificEnergy := PumpPower / Flow;
ELSE
```

```
    SpecificEnergy := 0;
  END_IF;

  Overflowing := Inflow > Flow AND WellLevel > 80;
```

Supported: `:=`, `+ - * /`, `MOD`, comparisons `= <> < > <= >=`, `AND OR XOR NOT`, `TRUE FALSE`, `IF … ELSIF … ELSE … END_IF`, `CASE x OF 1: … 2, 3: … 4..9: … ELSE … END_CASE`, `FOR i := 1 TO n BY step DO … END_FOR` (the step may be negative), `WHILE … DO … END_WHILE`, `REPEAT … UNTIL … END_REPEAT`, `EXIT` (leave the loop), `RETURN` (end the script) and comments `(* *)`. Names are not case-sensitive.

Function blocks keep their state from one run to the next (starting the loop starts them from scratch). Declare each one in a `VAR … END_VAR` block at the top of the script, then call it by its name:

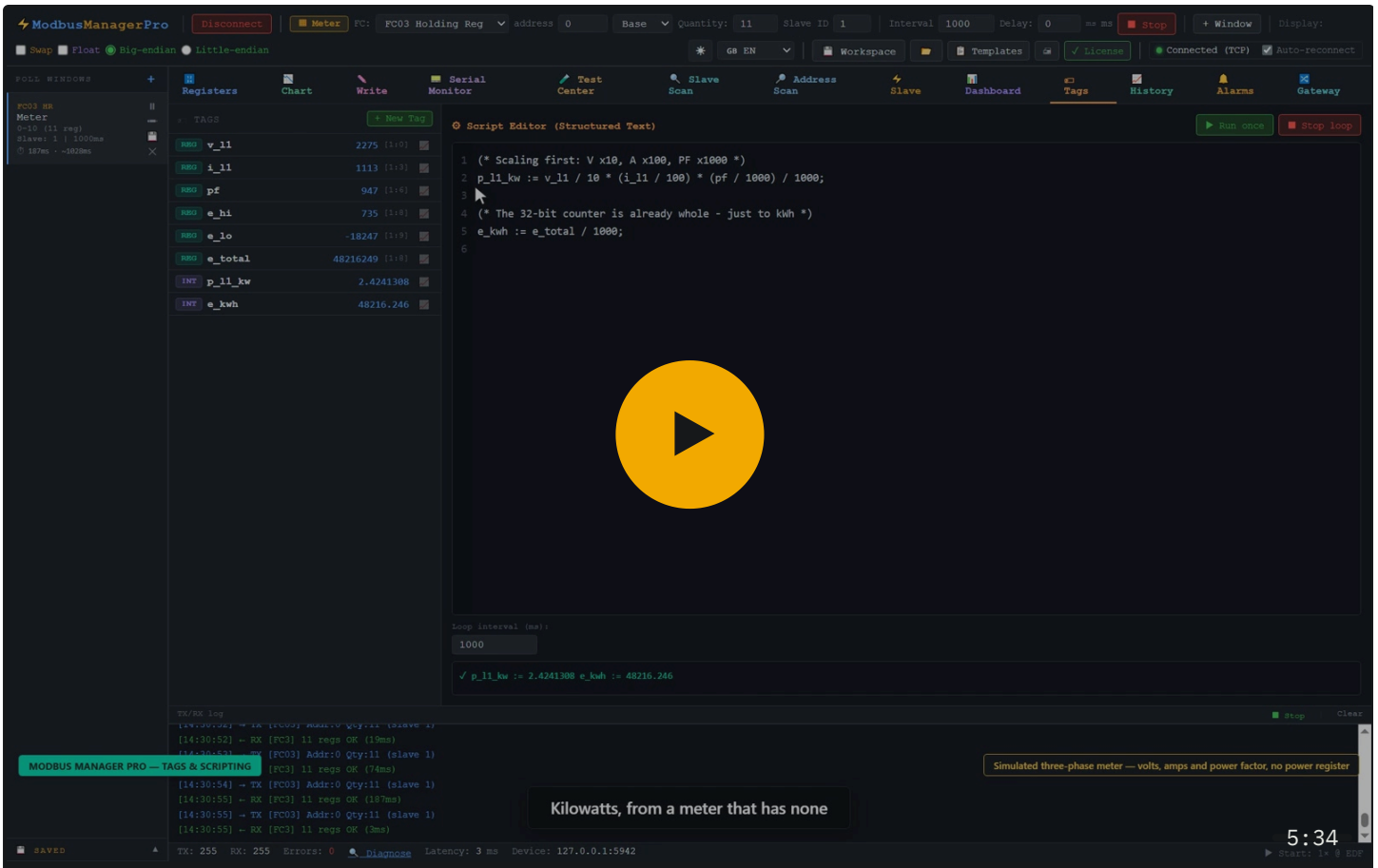
- **Timers**: `TON` (on-delay: Q once IN has been TRUE for PT), `TOF` (off-delay), `TP` (a pulse of exactly PT). Times are written `T#5s`, `T#500ms`, `T#1m30s`; ET written to a tag is in milliseconds. A timer is only as exact as the loop interval.
- **Edges**: `R_TRIG`, `F_TRIG` — Q for one run when CLK rises or falls.
- **Counters**: `CTU`, `CTD`, `CTUD` — count rising edges; R resets, LD loads PV.
- **Latches**: `SR` (set wins), `RS` (reset wins).

```
VAR
  StartDelay : TON;
  Edge       : R_TRIG;
  Starts     : CTU;
END_VAR

(* motor runs 5 s after the command *)
StartDelay(IN := RunCmd, PT := T#5s);
Motor := StartDelay.Q;

(* count the starts *)
Edge(CLK := Motor);
Starts(CU := Edge.Q, R := ResetCount, PV := 1000);
StartCount := Starts.CV;
```

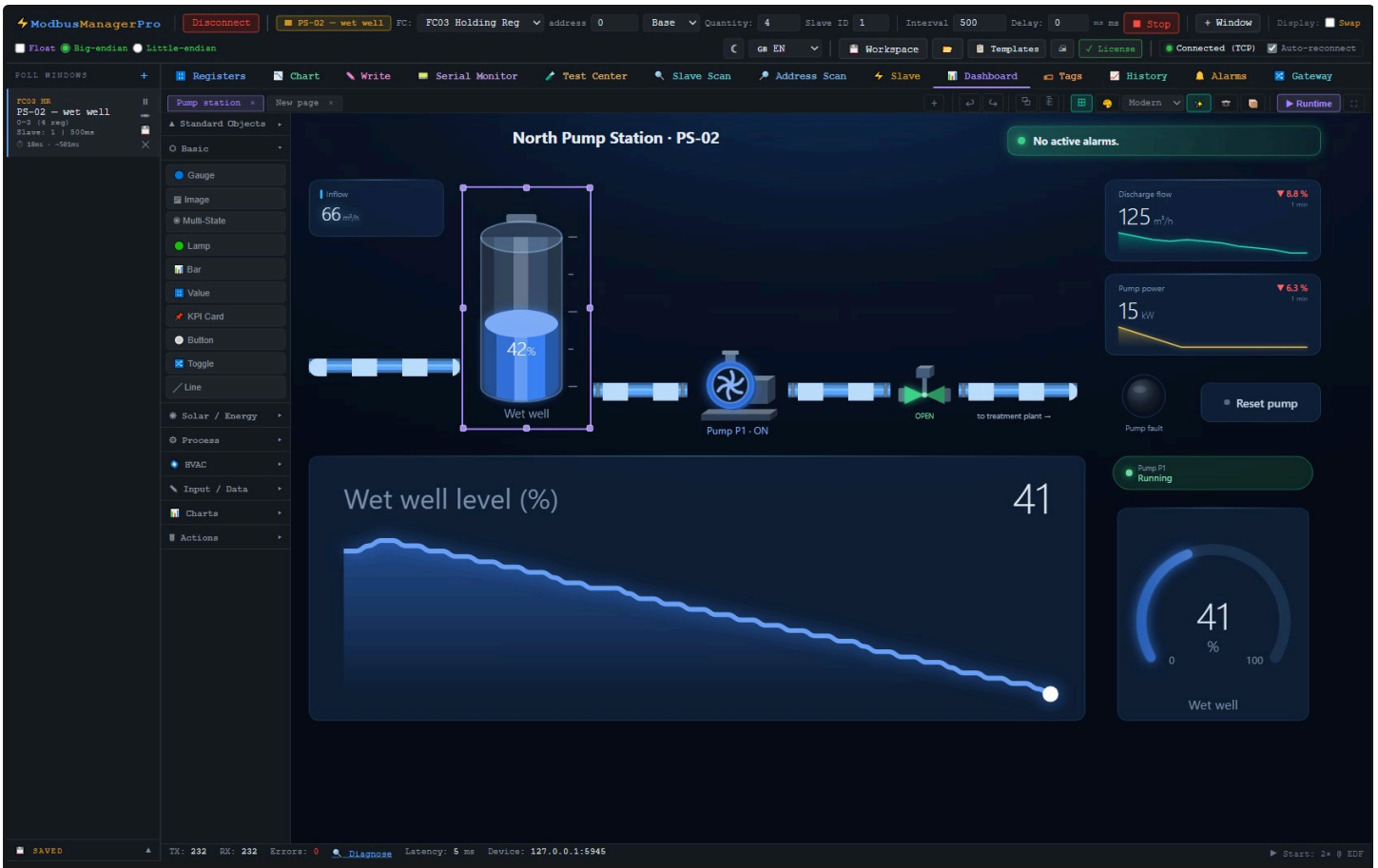
? Help in the editor lists every statement and block, with this example to copy.



Naming registers, a 32-bit energy counter read as one number, and a Structured Text loop producing kilowatts from a meter that reports none.

HMI DASHBOARD PRO

The **Dashboard** is a drag-and-drop canvas for operator screens. Pick an element from the palette, place it, bind it to a register, tag or S7 address in the **Properties** panel, then press ▶ **Runtime** to operate it.



PAGES AND TOOLBAR

- **Pages**: + adds one; double-click a tab for **Page settings** — name, size, and **Popup page** (hidden from the tabs, opened by clicking an element whose *Open popup* points to it). The tab's × deletes a page after asking; `Ctrl`+`Z` brings it back.
- **Undo / Redo** (`Ctrl`+`Z` / `Ctrl`+`Y`), **Copy / Paste** (`Ctrl`+`C` / `V`), snap to grid, background colour.
- **Theme**: **Classic** (flat) or **Modern** (cards, 3D equipment, glow). The theme changes only the look — no binding or setting of any element changes. In Modern, ✨ switches the glow and moving-value effects.
- 📷 exports the page as PNG; 📦 makes a Runtime package; ⛶ goes full screen (Esc leaves).
- Drag a frame to select several elements; arrow keys nudge; the **Actions** group aligns, locks, deletes and clears the page.

ELEMENTS

GROUP	ELEMENTS
Standard Objects	Rectangle, Rounded Rect, Group Frame, Circle, Ellipse, Polygon, Polyline, Circular Arc, Ellipse Arc, Pie Segment, Ellipse Segment, Static Text, Connector
Basic	Gauge, Image, Multi-State, Lamp, Bar, Value, KPI Card, Button, Toggle, Line
Solar / Energy	Semi Gauge, Battery, 3-Phase Meter, PV Energy Flow, PV Array, Inverter, Grid Connection, EV Charger, Heat Pump, Energy Accounting, Trend, Flow Arrow, Icon + Value, Progress Bar, Status Badge

GROUP	ELEMENTS
Process	Tank, Motor, Valve, Pump, Pipe, Silo, Conveyor
HVAC	Fan, Boiler, Damper, Heating Coil, Cooling Coil, Filter, Compressor, AHU, Thermostat, Humidifier, Heat Exchanger, Chiller, Rooftop Unit, VRF Indoor Unit
Input / Data	Slider, Enum Write, Setpoint, PID Loop, Clock
Refrigeration	Cold Room, Display Case, Compressor Rack
Weather	Wind Compass, Irradiance, Outdoor Air
Water	Reservoir, Wet Well, Flow Meter
Electrical	Circuit Breaker, Busbar, Transformer, Generator, Transfer Switch, 3-Phase Meter
Charts	Bar Graph, Value Table, Device Table, Status Bits, Indoor Unit, Motor / Drive, Annunciator, Multi Trend, Alarm List, Alarm Banner

PROPERTIES AND BINDING

Select an element to open **Properties**. **Source** is **Register** (slave ID, register type, address — polled in the background while polling runs, even without a poll window), **Tag**, or **Siemens S7**. The other fields follow the element: label, min/max, unit, decimals, font size, compare and threshold, on/off colours and labels, states (`0=Off:#8892a4,1=On:#00d4aa`), options for Enum Write, write value for a Button, flow and spin direction, image file, size, layer order. In the Modern theme a **Style (Modern)** block adds main and second colour, alarm and warning levels, arc span, glow and glass card. A **KPI Card** shows the change over a period, in per cent or in units.

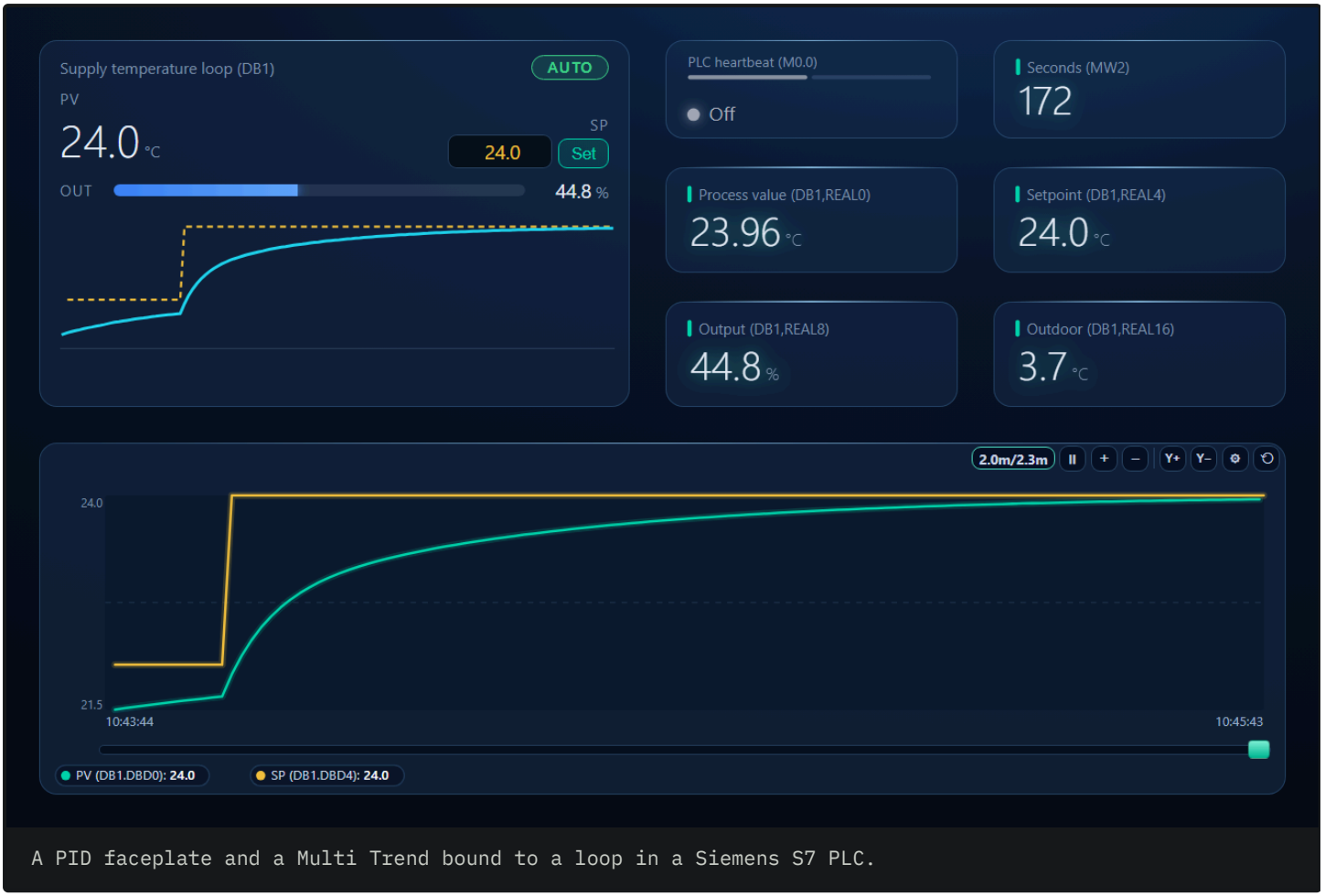
TRENDS AND CHANNEL CHARTS

- **Trend**: one value over time. **Max points** sets the length, **Sample interval (ms)** how often a point is taken (default 1000 ms; never faster than the value actually arrives). A steady value keeps drawing.
- A Trend or Multi Trend channel can take its **Source** → **History**: pick a series the Historian logs, and the chart draws what History holds and follows as new samples are stored — a page opened in the afternoon shows the morning too.
- A trend line breaks only where the data really stopped: a gap longer than 5 s, or three samples, whichever is longer. A short pause in polling no longer cuts the line.
- **Bar Graph, Value Table, Multi Trend, Alarm List** hold several channels: 📋 **Edit channels…** sets each channel's source (register or tag), name, unit, colour and, for Multi Trend and Value Table, **Visible when** (show the channel only while a tag is non-zero).
- **Multi Trend** adds max points, sample interval, **Scroll back (min)** up to 24 h, decimals, Y range and legend position. In Runtime it has pause, zoom on X and Y, a time-window dialog and a scroll-back slider.
- **Alarm List** uses manual thresholds or the alarm system; with the alarm system it, and the **Alarm Banner**, offer ✓ **Ack all**.

- A Value Table, Device Table, Alarm List or Alarm Banner that you have scrolled keeps its place while the values update.

PID LOOP FACEPLATE

The **PID Loop** element shows a controller: PV, setpoint, output bar, a small trend, auto/manual and the P, I and D values. Each role — Process value, Setpoint, Output, Auto/Manual, Manual output, P, I, D — is bound to a register or a tag, with a raw-divisor for PV/SP and for the output (e.g. 10 when the register holds ×10). In Runtime the operator can change the setpoint, switch auto/manual and set the manual output, each after a confirmation. P, I and D are only displayed.



A PID faceplate and a Multi Trend bound to a loop in a Siemens S7 PLC.

GROUP FRAME

A **Group Frame** draws a frame with a title around the elements of one unit (“AHU-1 supply”, “Pump 1”). It is always behind the other elements and only its title takes the mouse, so the elements inside stay clickable. Drag it by the title and every element wholly inside it moves along. The title sits on the border or on a bar, left or centred; line, fill, colour and corner radius are yours. An optional **alarm point** — a register bit, a register compared with a value, a tag or one of the app’s alarms — turns the frame red with ALARM (or amber with WARN) next to the title, so you see from across the room which unit has the problem; without a reading the chip shows “?”.

DEVICE TABLE

The **Device Table** shows many devices of one kind, one row each: 24 inverters behind a data logger, 16 fans on one bus. Set the **rows** once (how many, the first number, a name such as `Inverter {n}`) and how the unit ID and the address move from one row to the next (**Unit ID step**, **Address step**: a Huawei SmartLogger keeps every inverter on unit 0, one block every 25 registers). **Edit columns…** sets each column: register or tag (a tag name with `{n}` or `{nn}` takes the row number), data type (U16, I16, U32, I32, F32, also word-swapped), gain, decimals, shown as a number, a number with a bar or a status text, and a total under the column (sum, average, min or max). A fault and a warning rule colour the row; a row whose device stops answering is dimmed. A click on a header sorts, a click on a row opens that device's details. The table only reads.

A column whose value stops arriving dims on its own; the whole row dims only when everything it shows is old.

STATUS BITS

Status Bits shows each bit of a 16- or 32-bit status word by name, with what a set bit means: OK, Info, Warning or Fault. A profile fills the names: ABB Drives status word (ACS580, ACS880, ACH580), CiA 402 statusword, PROFIdrive ZSW1 (SINAMICS), Huawei SUN2000 State 1 and Alarm 1–3; **Custom** starts empty. The full view lists every named bit; the compact view only the set ones, or a text of your own ("No alarms") when none is set.

ANNUNCIATOR

The **Annunciator** is a grid of labelled tiles, as on a control panel. A tile is a register bit, a register compared with a value, a tag, or one of the app's alarms, with a severity: Alarm (red), Warning (amber) or Status (green, for states such as "Pump 1 running"). With **Flash until ACK** a new alarm flashes until **ACK**, then stays lit while it lasts; one that clears before ACK keeps flashing, so a short alarm is not missed. **Steady** only lights. ACK on a tile that follows an app alarm acknowledges it in the alarm system too, so the alarm list and the history see it. **Mark the first alarm** labels FIRST the tile that started a burst, until all are normal again. **Test** lights every tile for 3 seconds. **Fill from a status word profile** makes one tile per named bit of one register. A point without a reading is shown as unknown, never as normal.

INDOOR UNIT AND MOTOR / DRIVE FACEPLATES

As on the PID faceplate, each point is a role bound to a register or a tag, and a role left unbound is not drawn. The **Indoor Unit** faceplate (a VRF indoor unit or a fan-coil) shows on/off, room temperature, the setpoint that belongs to the mode, mode, fan speed, alarm, filter, thermo and occupancy; in Runtime it switches the unit on or off after a confirmation and sets the setpoint, mode and fan speed. The **Motor / Drive** faceplate shows the drive's state (FAULT with its code, HAND, RUNNING, STOPPED), frequency, current, power, the speed reference against the actual speed, ready, at-setpoint and warning lamps and run hours. Start and Stop ask first, Reset sends a pulse and goes back, – and + change the reference. **CMD ≠ FB** warns when the drive has

not followed a start or a stop within the set time. A yes/no point can be one bit of a status word and a number can be divided: the **ABB Drives profile** preset binds the control word, the reference (÷ 200) and the status word's bits on one unit ID in one click. In HAND, Start, Stop and the reference are blocked. **Write the command again every (ms)** is for a drive whose fieldbus watchdog counts only writes (for example ABB 58.15 = Cw / Ref1 / Ref2): while the drive runs, ModbusManager writes the run command and the reference again at that interval, and the faceplate says *Holding*. Stop, Reset, a fault, a lost link, opening another workspace or closing the window end it.

HVAC EQUIPMENT WITH VALUES

The **AHU**, **Chiller**, **Rooftop Unit** and **VRF Indoor Unit** have a **View** choice in Properties. **Symbol** is the small picture for an overview page. **With values** draws the unit larger, with each bound value in its own labelled box and a dotted line to the place where it is measured, so "Flow 45.2 °C" and "Return 32.1 °C" are never confused. As on the faceplates, each point is a role bound to a register or a tag: a yes/no point can be one bit of a status word, a number can be divided (÷ 10 when the register holds tenths), and a role left unbound is not drawn. The run state is the element's own source; a fault point, with an optional fault text, turns the state at the top red. In this view the decimal mark follows the app's language (22,4 in German), and when the link drops the values show — rather than the last reading. These elements only read.

- **AHU**: inlet and supply air temperature, airflow, and up to three coils (preheat, cooling, reheat), each with valve position and flow and return temperature at its own pipe.
- **Chiller**: chilled water flow and return, condenser water in and out, capacity and up to four compressors, each from its own point or following the run state. Without condenser points the chiller is drawn air-cooled, with fans on top.
- **Rooftop Unit**: supply, return and outside air temperature, the outside air damper (its blades follow the position), the mode by name from **Modes** (`1=Cool,2=Heat,3=Fan,4=Auto`) and up to four compressors.
- **VRF Indoor Unit**: room temperature, setpoint, fan speed and discharge air, with the mode and the fan speed by name from **Modes** and **Fan speeds**, written as on the Indoor Unit faceplate. The discharge air is blue when cooling and orange when heating. To switch the unit or change its setpoint, put the **Indoor Unit** faceplate next to it.

EVERY DEVICE: SYMBOL OR WITH VALUES, FAULT AND LIMITS

The Pump, Valve, Tank, Pipe, Motor, Fan, Damper, Heating Coil, Cooling Coil, Filter, Humidifier, Heat Exchanger, Boiler and Compressor have the same **View** choice as the AHU: **Symbol**, or **With values**, where each bound point (current, speed, temperatures, position…) gets its own labelled box. The element's own source stays as it was; a point not bound is not drawn.

- **Fault**: a fault point turns the symbol red with **FAULT**, and the fault text is its tooltip. **Fault to the alarm list** also raises it in the alarms.
- A **Tank** draws the limits that are set (HH, H, L, LL) and blinks the one passed. A **Valve** or **Damper** with a command and a feedback point shows a mismatch when they differ by more than the tolerance (5 %) for

longer than 10 s.

- **Value**, **Icon + Value** and **KPI Card** have a **Limits** section: warning and critical, high and low, with **Hysteresis** so a value on the limit does not flicker. Classic colours the number and frame and marks ▲ / ▼ before the label; Modern uses its level colours. **Limit crossing to the alarm list** raises them as alarms (warning, or critical).

BUTTON, SETPOINT AND LAMP

- **Button** modes: **Toggle** (0 ↔ 1), **Fixed value** (the same value on every press) and **Pulse** (on, then off after 100–10 000 ms). The pulse's release is written by the application itself, also if the window is closed mid-pulse. **Ask before writing** asks a question of your own first. A **Feedback (state)** point colours the button by the state read back.
- **Setpoint** has – / + buttons with a **Step**, and **Min** / **Max**. The typed or stepped value is a proposal until **Set** or Enter writes it.
- **Lamp** can show its state text or its value under it (**Text under the lamp**).

3-PHASE METER AND PV ENERGY FLOW

The **3-Phase Meter** shows voltages (phase and line), currents as bars up to the rated current, power, power factor, frequency and energy, with unbalance for voltage and current. Phases are coloured as IEC 60445 (L1 brown, L2 black, L3 grey; red-yellow-blue is an option). **Bind** fills every point for an **Eastron SDM630** (input registers, float32) or a **Schneider PM5000** (holding registers, float32, 4-quadrant power factor) at one unit ID. The meter reads 32-bit floats itself, so no tag is needed.

PV Energy Flow draws PV, grid, battery and house with animated flows. Powers are in W. **Bind** fills a **Deye hybrid** or a **Huawei SUN2000 with a meter**, with the signs as the makers state them. Without a house point the house is the balance: PV + bought – sold + discharged – charged, marked as calculated.

ELECTRICAL SINGLE-LINE ELEMENTS

Circuit Breaker (closed, tripped, withdrawn), **Busbar**, **Transformer** (load, temperature), **Generator** (running, fault, power), **Transfer Switch** (source 1 / 2, available, auto), and **PV Array**, **Inverter**, **Grid Connection**, **EV Charger** and **Heat Pump** (COP, flow and return, defrost) make a single-line diagram. An element is drawn live from its own voltage point, or else from the element it is **Fed by**; one not read yet is drawn dead. In Runtime a click on a breaker closes or opens it, after a confirmation, once its state is read — never while it is tripped or withdrawn.

ENERGY ACCOUNTING

Energy Accounting reads a meter's total energy counter (kWh) that the History logs, and shows the consumption per day, week, month or year in steps, the total, the cost with **Price per unit**, the peak demand with **Peak price per kW**, and the same part of the period before ("vs yesterday, same time"). A counter that is reset is handled. In Runtime the period can be changed for viewing only.

REFRIGERATION, WEATHER AND WATER

- **Cold Room** and **Display Case**: air temperature, setpoint, door, defrost, evaporator fan, cooling, light and night blind. **Too warm** counts only after its delay and not during a defrost or the time set after it, as HACCP records want; a door open too long is shown in seconds. **Compressor Rack**: suction and discharge pressure, suction temperature, capacity and the compressors running. These elements only read; **Alarms to the alarm list** raises their alarms.
- **Wind Compass** (direction where the wind comes from, speed and gust, in m/s, km/h, knots or mph with the Beaufort number), **Irradiance** (W/m², today's kWh/m², panel temperature) and **Outdoor Air** (temperature, humidity, dew point, pressure).
- **Reservoir** (level, inflow, outflow, overflow, HH/H/L/LL), **Wet Well** (level, the pumps with their start and stop levels as lines, pump faults) and **Flow Meter** (flow, totaliser, reverse flow).

The **Properties** panel can be made wider: drag its left edge (up to half the window); a double-click on the edge sets it back.

RUNTIME MODE AND WRITES

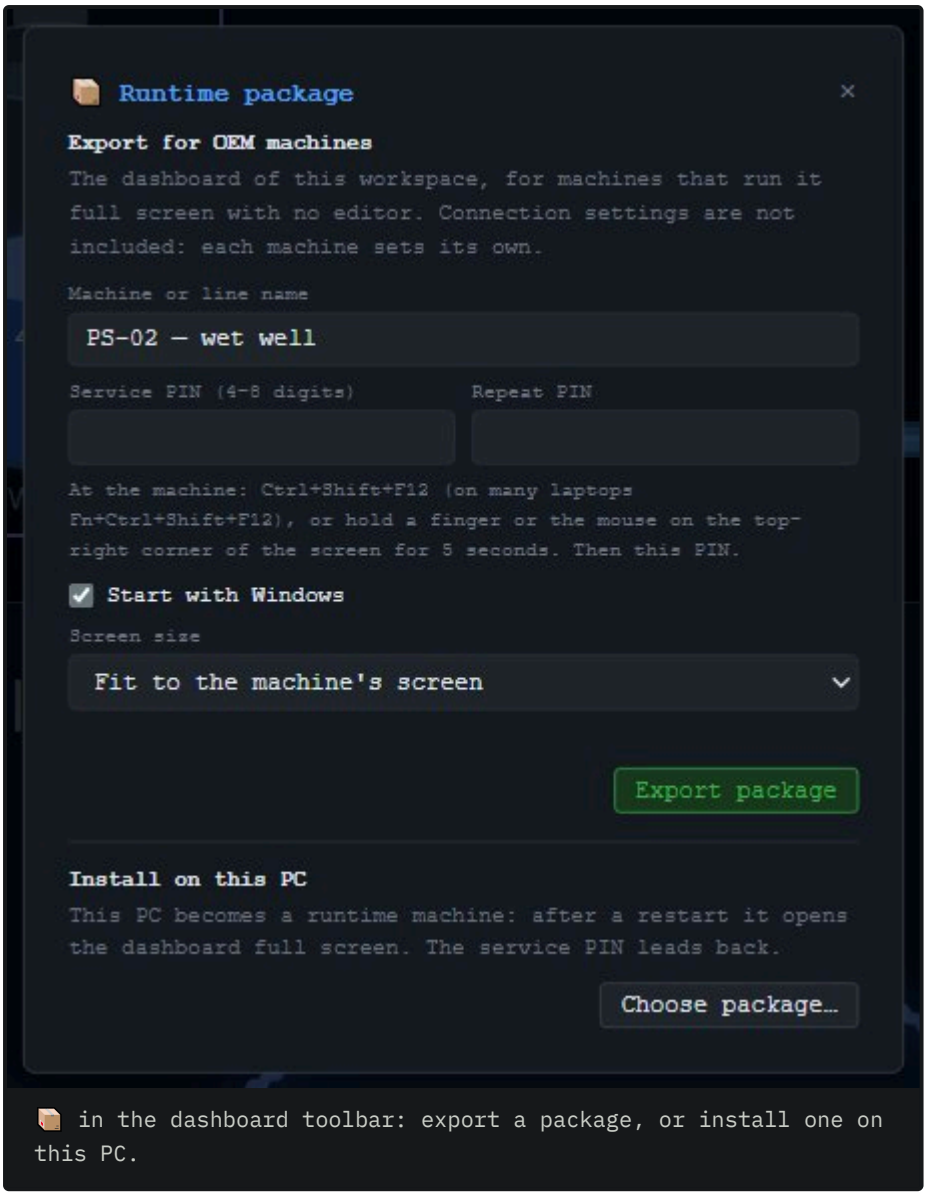


Runtime during a high-level alarm: the banner, the red tank and gauge, the stopped pump.

- Write elements (Button, Toggle, Slider, Setpoint, Enum Write, PID) write **only in Runtime**, and only after the element has read a value, so a click in edit mode or on an empty screen never sends anything.
- A value that stops arriving is dimmed and keeps its last reading; hover shows "Not current — last update …". Text in a numeric field shows "—".

RUNTIME EDITION PRO

A machine builder designs the screen once in Pro and installs it on every machine's panel PC, where it runs full screen with no editor. See also the HMI Runtime page.



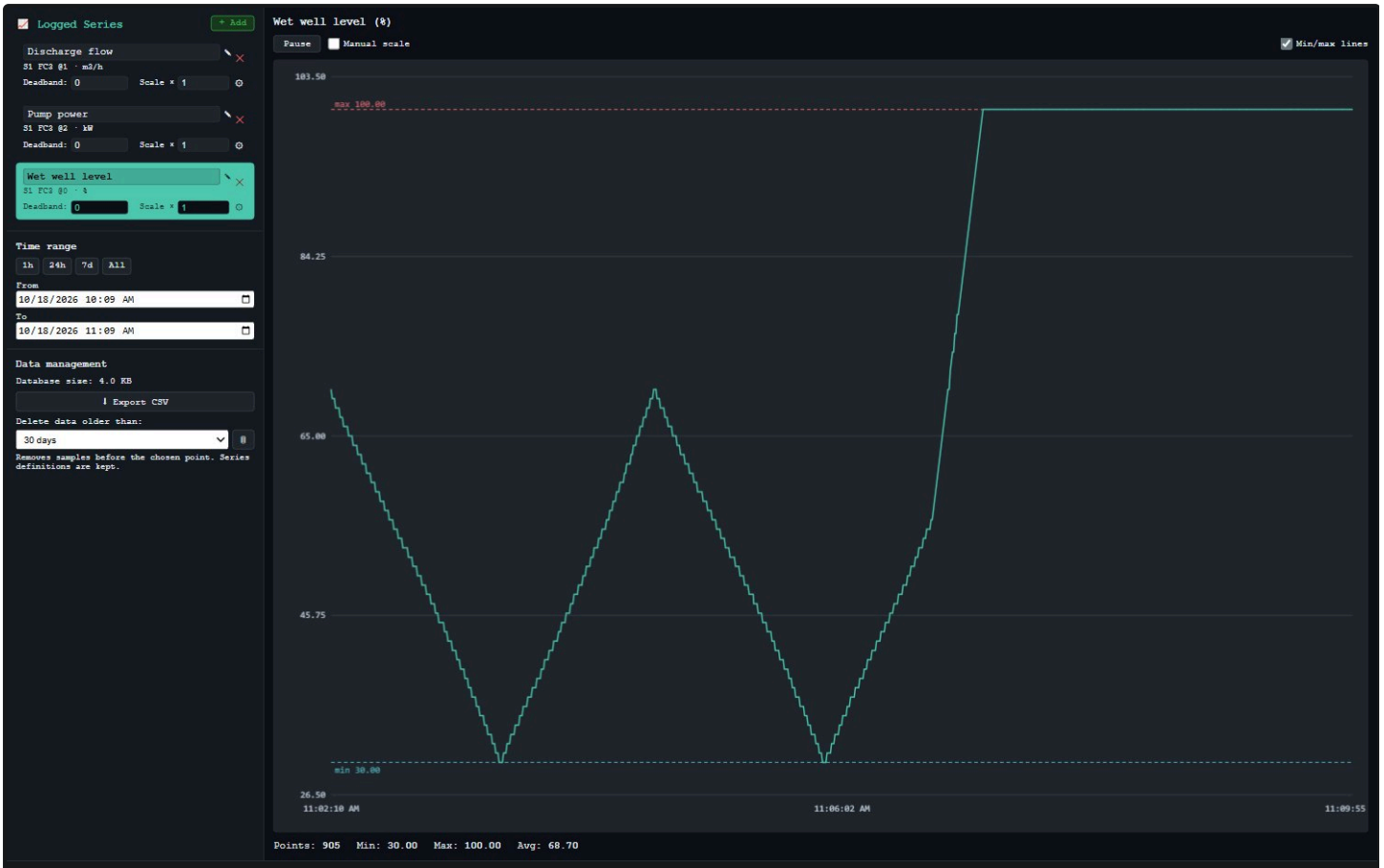
in the dashboard toolbar: export a package, or install one on this PC.

1. **Export**: name the machine or line, set a **Service PIN** (4–8 digits), choose *Start with Windows* and the screen size (*Fit to the machine's screen* or *Actual size 1:1*), then **Export package**. A `.mmrt` file is written to Documents. It carries the dashboard, tags and script — not the connection settings and not the gateway.
2. **Install**: on the panel PC open → **Choose package…** and restart. The PC opens the dashboard full screen; on the first start it asks for this machine's connection (and S7 PLC addresses).
3. **Service**: press `Ctrl` + `Shift` + `F12` (on many laptops with `Fn`) or hold the top-right corner for 5 s, then the PIN. The Service menu has Connection settings, Siemens S7 PLCs, Install another package, Licence, Open the editor (only with a Pro licence or trial), Make this an ordinary PC, and Close the application. Five wrong PINs lock it for a minute.

Licensing: a Runtime key (`MBUSR-`, $39 per machine) runs packages without limit but never opens the editor. A PC with a Pro licence or trial runs them too. Without either, a package runs as a 2-hour demo: then polling stops and the Machine ID is shown with an Activate button; a restart gives another 2 hours.

HISTORY (HISTORIAN) PRO

The Historian stores values in a local database for days or months and charts any time range.



Logged series on the left, one hour of the wet well level on the right.

- **Start logging**: 📈 on a register row or a tag, or **+ Add** in **Logged Series** (name, register or tag, slave, function, address, data type, word order, unit). Recording starts at once.
- Samples arrive at the poll rate. **Deadband** stores a sample only when the value moved at least that much (0 = store all); writes to disk are batched every 5 s.
- **Scale ×** and ⚙ **Display scale** (scale, offset, decimals, chart min/max) change how a series is shown; the database keeps the raw value, so a scale can be corrected later.
- **Time range**: 1h, 24h, 7d, All, or From/To. Drag on the chart to zoom, **Manual scale** fixes the Y axis, **Min/max lines** marks the extremes; the statistics show points, min, max and average.
- **Data management**: database size, **⬇ Export CSV** of the selected series (ISO time, milliseconds, value), and **Delete data older than** 1 day … 1 year or a date. Series definitions are kept.

More on intervals and deadband: logging interval & deadband · CSV export.

ALARMS PRO



An active critical alarm waiting for acknowledgement, and two definitions.

- **+ New alarm**: name, description, source (register or tag) and condition: greater, less, greater or equal, less or equal, equal, not equal, between, outside range.
- Greater/less conditions have two optional levels, ⚠ **Warning limit** and ⛔ **Critical limit**; equal/not equal take a value; range conditions take lower and upper.
- **Hysteresis** keeps a value that hovers at the limit from raising and clearing the alarm repeatedly; **Delay (s)** raises it only after the condition has held that long.
- **Active alarms** lists what is raised; ✓ **Ack** acknowledges. **Sound on** / **Muted** and *Repeat until ack* / *Beep once* choose the alert. **History** lists raised, cleared and acknowledged times.
- On a dashboard, the **Alarm Banner** and an **Alarm List** in alarm-system mode show the same alarms and acknowledge them.

NOTIFICATIONS: E-MAIL, TELEGRAM, WEBHOOK

📧 **Notifications** next to **Active alarms** sends alarms out: by **e-mail** (your SMTP server, e.g. Gmail with an app password), to a **Telegram** group (a bot made with @BotFather; **Find chat ID** fills the chat after one message in the group) or to a **webhook** (HTTP POST with JSON — ntfy, Home Assistant, Node-RED, n8n, Slack or Teams workflows). Each channel has a **test** button and a status line.

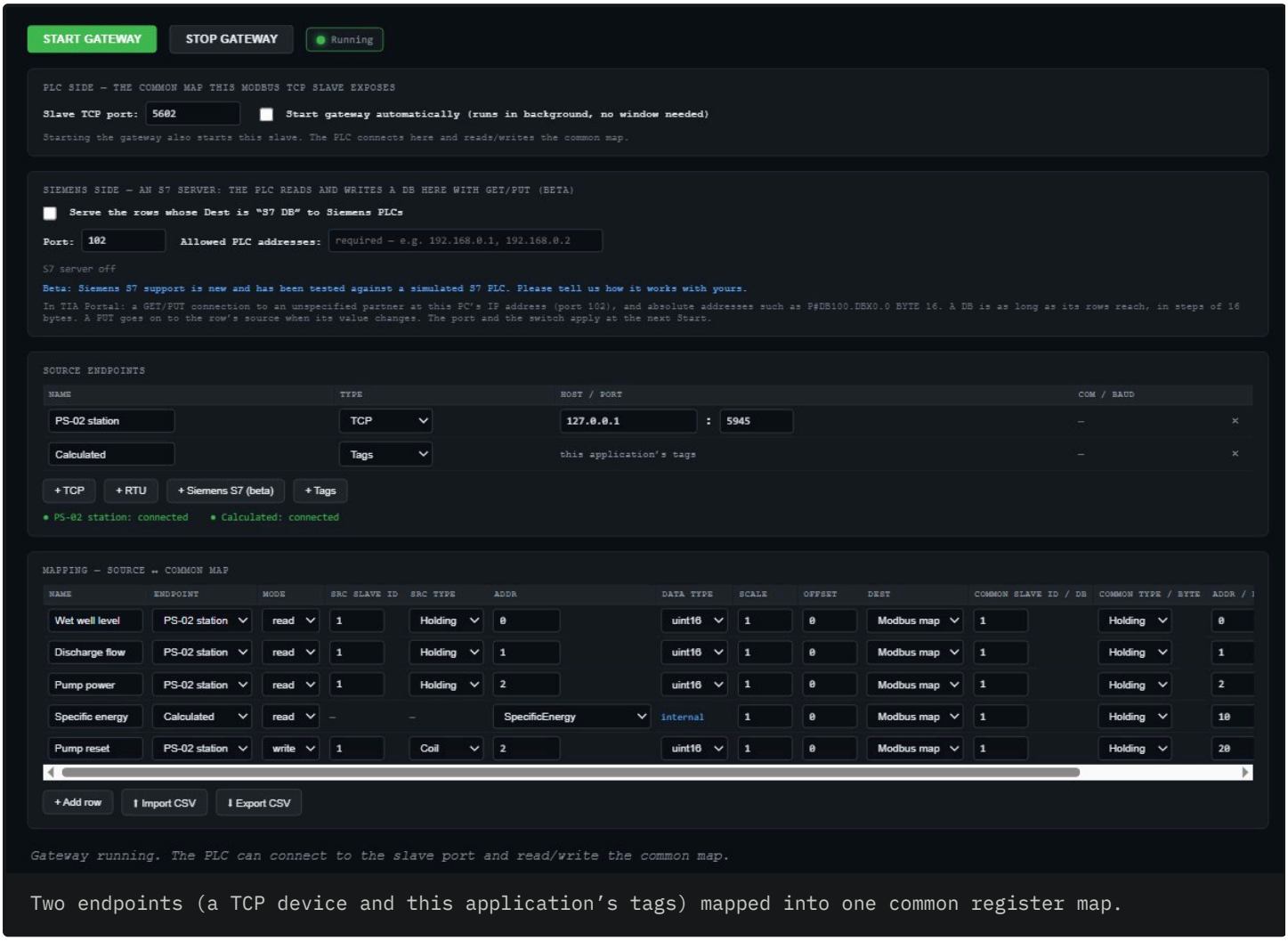
- **When to send**: alarm raised, warning → critical, back to normal, acknowledged; the lowest level; the same alarm again only after a number of minutes (a flapping alarm sends once); and above a number of messages a minute, one message lists them all (a PLC going offline raises many at once).
- Each alarm has a **Notify** box (on by default). Dashboard element faults and limits that go to the alarm list are sent too.
- The messages are in the application's language and name the site, the alarm, the value and the limit. A failed send is tried again three times, a minute apart.

- The application sends them, so they work with the window closed and on a Runtime machine. Passwords and tokens stay on the computer: they are not saved in the workspace or in a Runtime package — a Runtime machine gets the settings with its package and its own passwords through the service menu.

> NOTE
> Alarms are evaluated and notifications sent by the application while it runs — keep it running in the tray (*Keep running when window is closed*) for unattended monitoring.

GATEWAY PRO

The gateway polls several devices as a master and republishes the values you choose as one **common Modbus TCP map** — or as a data block an S7 PLC reads with GET, or to an **MQTT** broker such as Home Assistant's — with type and scale conversion both ways. Writes from the PLC (or from Home Assistant) are forwarded to the device.



Two endpoints (a TCP device and this application's tags) mapped into one common register map.

1. **PLC side**: the **Slave TCP port** the PLC or SCADA connects to (502 by default).

2. **Source endpoints**: **+ TCP** (host, port), **+ RTU** (COM port, baud, parity), **+ Siemens S7** (a PLC from the S7 list) and **+ Tags** (this application's tags, including script results).
3. **Mapping rows**: name, endpoint, mode (**read** device → map, or **write** map → device), source slave ID, type and address (an S7 address or a tag for those endpoints), data type, scale and offset, then the destination: **Modbus map** (common slave ID, type, address, data type), **S7 DB** (DB, byte, bit, S7 type) or **MQTT** (topic, unit, Home Assistant device class).
4. **Start gateway**. Each endpoint and row shows its state; a row with an error says why.

⬆ Import CSV and **⬇ Export CSV** move rows and endpoints to and from a spreadsheet. **Start gateway automatically** plus the tray options *Keep running when window is closed* and *Start with Windows* run it unattended. The gateway configuration is saved with the workspace.

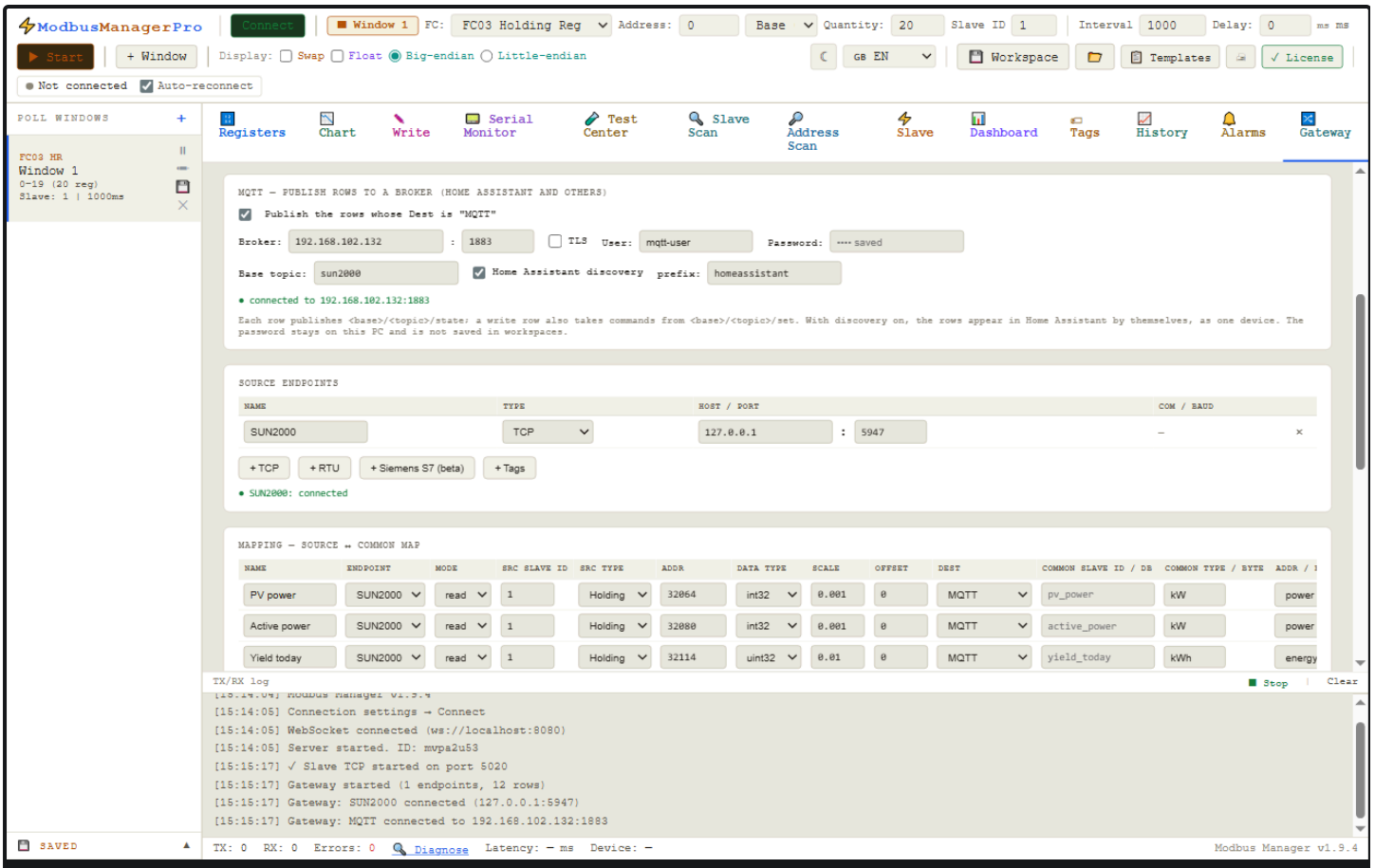
S7 SERVER: THE PLC READS A DB FROM MODBUSMANAGER

With **Serve the rows whose Dest is "S7 DB" to Siemens PLCs** the gateway also answers as an S7 server (port 102): an S7-1200/1500 reads the rows with a **GET** block and writes them with **PUT** — no Modbus program in the PLC. In TIA Portal create a GET/PUT connection to an unspecified partner at this PC's IP address and use absolute addresses such as `P#DB100.DBX0.0 BYTE 16`. A DB is as long as its rows reach, in steps of 16 bytes; a PUT is forwarded to the row's source when its value changes.

> ALLOWED PLC ADDRESSES
> Only PLCs listed in **Allowed PLC addresses** (comma-separated IPs) can connect. An empty list refuses everyone, and the panel warns about it. The list applies at once; removing an address drops that PLC.

MQTT AND HOME ASSISTANT

A row with Dest **MQTT** publishes its value to an MQTT broker — Home Assistant's Mosquitto add-on or any other — as `<base topic>/<topic>/state`, retained. A 32-bit float is published with 7 significant digits (21.8, not 21.79999924). The row keeps its own source, scale and offset; only the destination differs, so a device can feed the common map, an S7 DB and MQTT at the same time from one connection.



The MQTT panel connected to Home Assistant, and rows with Dest MQTT.

1. In the **MQTT** panel tick **Publish the rows whose Dest is "MQTT"**, then set the **Broker** address and port (1883; 8883 with **TLS**), **User**, **Password** and the **Base topic**.
2. Set a row's **Dest** to **MQTT**. Its columns become the topic (empty: made from the name), the unit and the Home Assistant device class; the last column shows what the row becomes in Home Assistant.
3. **Start gateway**. The panel says *connected*, or why not: user name or password refused, no broker at this address, broker not reachable, certificate not trusted.

With **Home Assistant discovery** on, each MQTT row appears in Home Assistant by itself, all under one device, with its name, unit and device class: a value as a *sensor* (a coil or discrete input as a *binary sensor*), a write row as a *number* (a coil as a *switch*). Rows with device class *energy* are published as growing totals for the Energy dashboard. A write row also takes commands from `<base>/<topic>/set`: the value goes to the device and is read back. **Publish alarms** sends every alarm event to `<base>/alarms/event` and keeps each active alarm retained at `<base>/alarms/<id>`, removed when it clears. `<base>/status` says *online* or *offline*, so Home Assistant shows the entities as unavailable when the gateway stops or the PC drops off. A row deleted here is removed from Home Assistant too.

> THE PASSWORD STAYS ON THIS PC
>
> The broker password is stored apart from the gateway configuration and is never written to a workspace or a CSV export. A workspace moved to another PC asks for it again. Home Assistant's Mosquitto add-on accepts a Home Assistant user name and password.

Full example with a free workspace: Huawei SUN2000 to Home Assistant over Modbus and MQTT.

SIEMENS S7 PRO

Pro reads and writes Siemens S7-1200, S7-1500 and S7-300/400 PLCs over S7 communication (PUT/GET) — data blocks, merkers, inputs and outputs — with no communication block in the PLC program. It has been tested against a real S7 PLC and is no longer marked beta.

1. In TIA Portal: CPU properties → Protection & Security → *Permit access with PUT/GET communication*; for every DB you read, turn off *Optimized block access*; download.
2. In ModbusManager: Tags → **Siemens S7…** (also in the gateway, the dashboard source and the Runtime service menu) → **+ Add PLC**: ID, name, IP address, port 102, rack 0 and slot 1 for S7-1200/1500 (slot 2 for S7-300/400), poll interval. **Test read** checks an address.
3. Create S7 tags, bind dashboard elements to **Siemens S7**, or add an S7 endpoint to the gateway.

Address syntax: `DB1,REAL4` · `DB1,INT2` · `DB1,X12.0` (bit) · `MW2` · `M0.0` · `IW64`. Writes are converted to the address type (a bit takes 0/1, integers are rounded, REAL keeps decimals). The PLC's state shows as online, connecting, error, or idle when nothing reads from it. The PLC ID ties tags to the PLC, so a runtime package keeps working when each machine's PLC has a different IP.

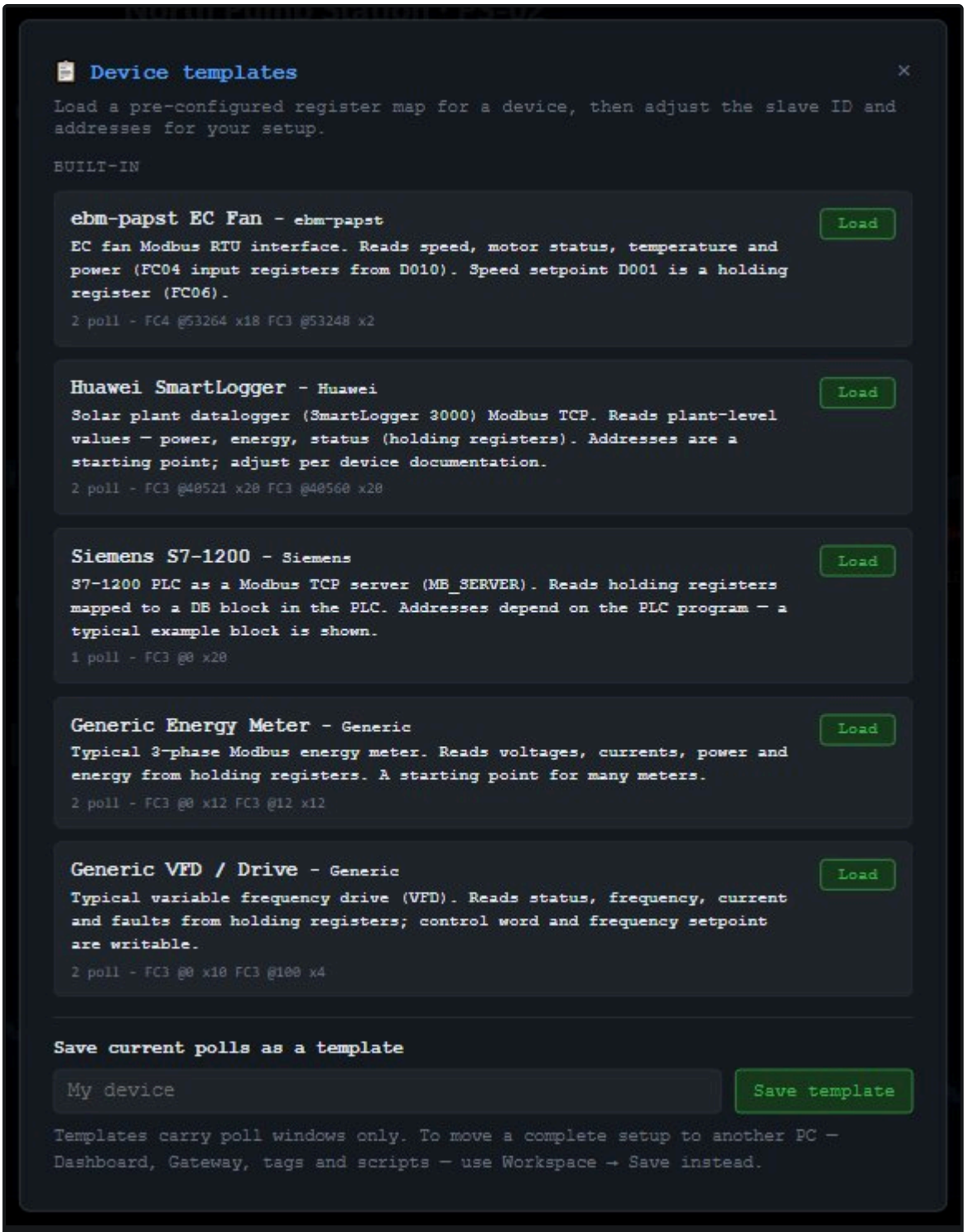
Full guide: Siemens S7-1200 / S7-1500 PUT/GET from a PC.

WORKSPACES & TEMPLATES

A **workspace** `STD + PRO` is one JSON file with your setup. 💾 **Workspace** saves it to Documents; the folder button opens one from the list or by **Browse…**. On quit the application asks whether to save.

SAVED IN THE WORKSPACE	NOT IN THE WORKSPACE
Poll windows and their descriptions, Value Names, conditional colours, chart series, slave blocks; in Pro also the dashboard (pages, theme, effects, background), tags, the script and its interval, which series the Historian logs, and the gateway configuration (opening asks before applying it).	Connection settings (each PC keeps its own), alarm definitions and the S7 PLC list (kept by the application), simulations on slave registers, and recorded history.

Device templates `PRO` load ready poll windows for a device — ebm-papst EC fan, Huawei SmartLogger, Siemens S7-1200 (as Modbus TCP server), a generic energy meter and a generic VFD — and **Save template** stores your own. Saving under a name that exists asks, then replaces that template. A templates file that cannot be read is set aside as `device-templates.damaged-<time>.json` and reported, never overwritten. Templates carry poll windows only; use a workspace to move a complete setup.



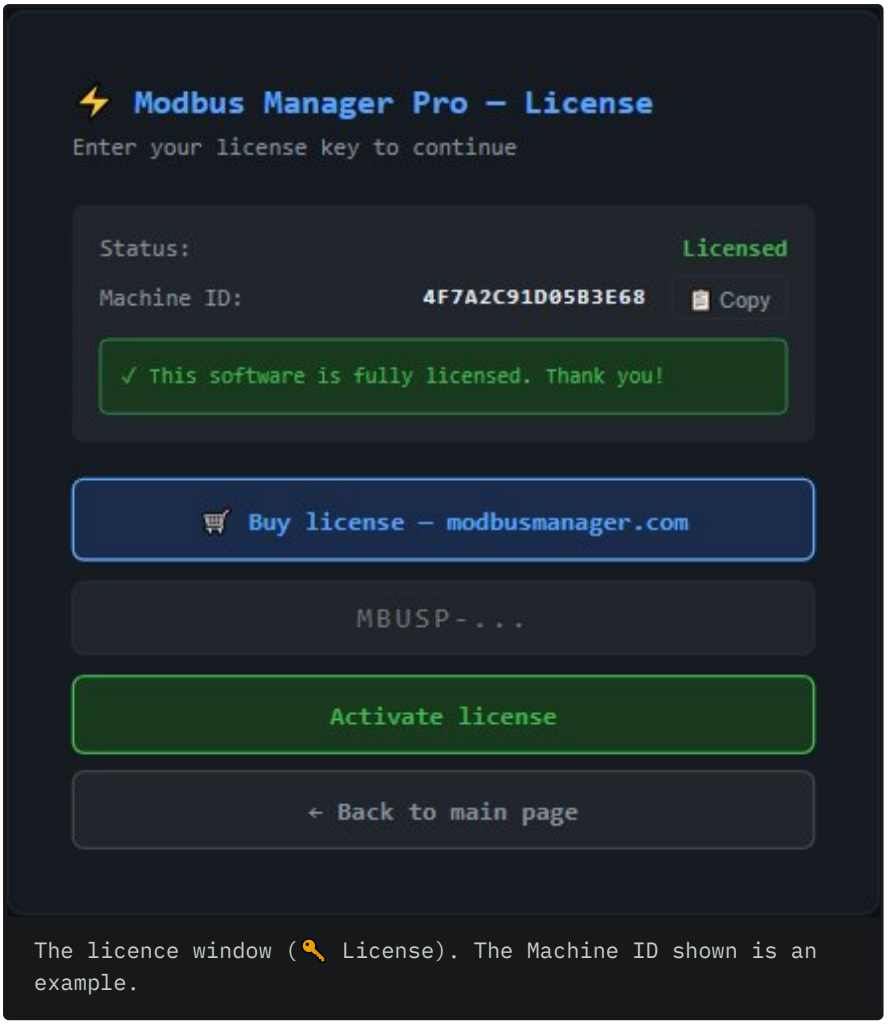
Device templates.

Ready workspaces for many devices are on the device guide pages (Guides menu), e.g. Eastron SDM630 or Huawei SUN2000.

Device library PRO lists those workspaces inside the application, with a search and categories (energy meters, solar, drives, HVAC, pumps, PLC, BACnet network, Home Assistant). **Open** shows what the workspace holds — pages, elements, poll windows, tags, whether it has a script — and that it brings no connection, and asks before it replaces yours (**Save mine first…**). **Open in ModbusManager** on a device guide page does the same from the browser.

If a workspace has tags that cannot be created (for example a reserved name such as `In`), a window lists each one with the reason, the dashboard elements and script lines that use it, and how to fix it; the rest of the workspace opens as usual.

LICENSING



The licence window (🔧 License). The Machine ID shown is an example.

- **Trial**: 14 days with every feature, from the first start. Reinstalling does not reset it. When it ends, the licence window opens instead of the application until a key is activated.
- **Machine ID**: shown in the licence window, with a copy button. A key is issued for that ID: buy on modbusmanager.com, enter the Machine ID, and the key arrives by e-mail.
- **Activate**: paste the key (Pro keys start `MBUSP-`, Standard `MBUS-`, Runtime `MBUSR-`) and press **Activate license**. The licence is perpetual for that machine.
- **New computer**: a key is bound to one machine; write to support@modbusmanager.com to move it.
- Updates are free for 12 months from purchase; see the licence terms.

LANGUAGES

The interface is translated into nine languages, chosen from the flag in the toolbar. **Pro**: English, German, Spanish, French, Portuguese, Ukrainian, Chinese, Polish and Estonian. **Standard**: the same with Russian in place of Ukrainian. A restart applies the choice everywhere.

TROUBLESHOOTING

SYMPTOM	CHECK
No reply from a TCP device	IP and port (502), that the PC reaches the network (ping), the unit ID, and whether another master is holding the device's only connection.
RTU: timeouts	COM port, baud, parity, data and stop bits; A/B polarity of RS-485; termination on long lines; the slave ID. Try **Auto-detect**.
Exception 02 (illegal address)	Address base (0 or 1), function code (FC03 vs FC04), and that the block does not run past the device's map — read fewer registers.
32-bit values look absurd	Word order: right-click the 32-bit cell and try the four orders, or set *Low word first* on the tag.
A dashboard element shows "—" or is dimmed	Polling is stopped, the source is wrong, or the value stopped arriving. Hover it for the last update; check the binding in Properties.
A write does nothing	The dashboard must be in **Runtime**, and the element must have read a value first. Input registers and discrete inputs cannot be written.
S7: connection refused or no data	PUT/GET permitted, optimized block access off for the DB, rack/slot, and for the S7 server that the PLC's IP is on the allowed list.
Trial expired	Activate a key for your Machine ID in the licence window.

The 🔍 **Diagnose** link and **Auto-detect** (see Scanning) answer most connection questions before a support e-mail. If you write to support, copy the diagnostic text from the licence reminder's *I am stuck* page.

FAQ

How do I connect to a Modbus TCP device?

Press Connect, stay on TCP/IP, enter the IP address and port (502 by default) and connect; then set the slave/unit ID in the poll window and press Start.

Which function codes are supported?

Reading: FC01, FC02, FC03, FC04. The Write tab writes FC05, FC06 and FC16. The Test Center sends any single request, including FC15, FC22 mask write, FC23 read/write, FC08 diagnostics, FC11, FC17 and FC43/14 device identification.

What is the difference between address base 0 and base 1?

Base 0 uses protocol addresses (first register = 0); Base 1 uses documentation addresses (first register = 1). Pick the one your device manual uses.

Can I test without hardware?

Yes. The Slave tab simulates one or more devices over TCP or RTU, with counters, sine waves, ramps, random values and a simulated PID process.

What is the difference between Standard and Pro?

Standard ($49) is the master/slave test tool. Pro ($119) adds the HMI dashboard, tags and Structured Text, Historian, alarms, gateway, Siemens S7 and Runtime packages.

Is there a free trial?

Yes, 14 days with every feature and no credit card. After that you activate a key issued for your machine.

Windows shows “Windows protected your PC” when I run the installer — is that a problem?

No. The installer is code-signed, but SmartScreen shows this notice for releases it has not seen many times yet. Click **More info**, then **Run anyway**. Every release is published on our GitHub releases page, so you can check the download comes from the official source.
